

Constructors & Destructors in Java

A Comprehensive Guide

Slide 1: Introduction

What are Constructors and Destructors?

- **Constructor:** Special method called when an object is created
 - **Destructor:** In Java, the `finalize()` method (deprecated since Java 9)
 - Java constructors initialize object state
 - Java uses garbage collection (no explicit destructors)
 - Memory management is handled automatically by JVM
-

Slide 2: Basic Constructor Syntax

Constructor definition:

```
public class Student {  
    public Student() {  
        System.out.println("Student object created!");  
    }  
}
```

Key rules:

- Constructor name must match class name
 - No return type (not even `void`)
 - Cannot be `abstract`, `static`, `final`, or `synchronized`
 - Called automatically when using `new`
-

Slide 3: Creating Objects

The `new` keyword:

```
public class Main {  
    public static void main(String[] args) {  
        Student s = new Student(); // Constructor called here  
    }  
}
```

What happens when `new Student()` executes:

1. Memory is allocated on the heap
 2. Fields are initialized to default values (0, null, false)
 3. Constructor body executes
 4. Reference is returned to the caller
-

Slide 4: Parameterized Constructors

Constructors with arguments:

```
public class Student {
    private String name;
    private int age;

    public Student(String name, int age) {
        this.name = name;
        this.age = age;
    }

    public void display() {
        System.out.println(name + " is " + age + " years old");
    }
}

// Usage
Student s = new Student("Alice", 20);
s.display(); // Alice is 20 years old
```

Slide 5: Constructor Overloading

Multiple constructors in a class:

```
public class Student {
    private String name;
    private int age;

    // Constructor 1: No-arg
    public Student() {
        this.name = "Unknown";
        this.age = 18;
    }

    // Constructor 2: Name only
    public Student(String name) {
        this.name = name;
        this.age = 18;
    }

    // Constructor 3: Full info
```

```
public Student(String name, int age) {  
    this.name = name;  
    this.age = age;  
}
```

- Constructors are differentiated by parameter lists
- Compiler determines which constructor to call

Slide 6: Understanding `this` Keyword

Using `this` in constructors:

```
public class Point {  
    private int x;  
    private int y;  
  
    public Point(int x, int y) {  
        this.x = x; // this.x = parameter x  
        this.y = y;  
    }  
  
    // this() for constructor chaining  
    public Point() {  
        this(0, 0); // Calls Point(int, int)  
    }  
  
    public Point(int x) {  
        this(x, 0); // Calls Point(int, int)  
    }  
}
```

Rules for `this()`:

- Must be the first statement in the constructor
- Can only be used once per constructor
- Enables constructor reuse (DRY principle)

Slide 7: Constructor Chaining

Reusing constructors efficiently:

```
public class Rectangle {  
    private int width;  
    private int height;  
  
    // Master constructor
```

```
public Rectangle(int width, int height) {
    this.width = width;
    this.height = height;
    System.out.println("Rectangle " + width + "x" + height);
}

// Delegates to master constructor
public Rectangle(int side) {
    this(side, side); // Square
}

public Rectangle() {
    this(1, 1); // Unit square
}
}
```

Slide 8: Default Constructor

When Java provides one automatically:

```
// No constructor defined
public class SimpleClass {
    private int value;
}

// Java implicitly provides:
// public SimpleClass() { }
// Fields get default values: value = 0
```

When default constructor is NOT provided:

- If any constructor is explicitly defined
- Example: If you define `Point(int x, int y)`, then `new Point()` won't compile

Slide 9: The `finalize()` Method

Java's approach to destruction:

```
public class Resource {
    private String name;

    public Resource(String name) {
        this.name = name;
        System.out.println(name + " created");
    }

    @Override
```

```
protected void finalize() throws Throwable {  
    System.out.println(name + " finalized");  
    super.finalize();  
}  
}
```

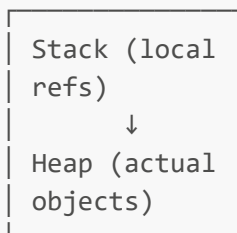
Important: `finalize()` is **deprecated** since Java 9!

- Unpredictable execution time
- May never be called
- Performance overhead
- Not a replacement for destructors

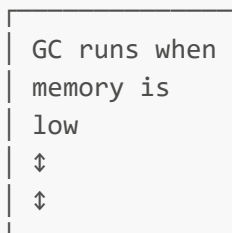
Slide 10: Why No Destructor in Java?

Java's garbage collection model:

Allocation:



Deallocation:



- GC runs automatically (non-deterministic)
- Objects may be collected in any order
- Cannot predict when `finalize()` will run
- C++ destructors are deterministic, Java GC is not

Slide 11: `finalize()` Lifecycle

Object finalization sequence:

```
public class Demo {  
    @Override  
    protected void finalize() {  
        System.out.println("Finalized");  
    }  
  
    public static void main(String[] args) {  
        Demo d = new Demo();  
        d = null; // Eligible for GC  
  
        System.gc(); // Suggest GC (not guaranteed)  
    }  
}
```

```
        // Output may or may not include "Finalized"
        System.out.println("End of main");
    }
}
```

Order of events:

1. Object becomes unreachable
2. GC marks it for collection
3. `finalize()` is called (at most once)
4. Object is reclaimed

Slide 12: `finalize()` Resurrection

Objects can escape finalization:

```
public class Phoenix {
    private static Phoenix savedRef = null;

    @Override
    protected void finalize() {
        System.out.println("Rising from ashes!");
        savedRef = this; // Object becomes reachable again!
    }

    public static void main(String[] args) {
        Phoenix p = new Phoenix();
        p = null;
        System.gc();

        // Object is still alive!
        Phoenix resurrected = Phoenix.savedRef;
        System.out.println(resurrected != null); // true
    }
}
```

- `finalize()` can make object reachable again
- Object will not be collected until next GC cycle
- `finalize()` runs only once per object

Slide 13: Cleanup with `AutoCloseable`

Modern Java alternative to destructors:

```
public class DatabaseConnection implements AutoCloseable {
    private String url;

    public DatabaseConnection(String url) {
        this.url = url;
        System.out.println("Connected to " + url);
    }

    public void query(String sql) {
        System.out.println("Executing: " + sql);
    }

    @Override
    public void close() {
        System.out.println("Closing connection to " + url);
        // Release resources here
    }
}
```

Slide 14: Try-with-Resources

Automatic resource management:

```
public class Main {
    public static void main(String[] args) {
        // Resource automatically closed
        try (DatabaseConnection db = new
DatabaseConnection("jdbc:mysql://localhost")) {
            db.query("SELECT * FROM users");
        } // close() called automatically here
    }
}

// Output:
// Connected to jdbc:mysql://localhost
// Executing: SELECT * FROM users
// Closing connection to jdbc:mysql://localhost
```

- `AutoCloseable` interface defines `close()`
- Try-with-resources guarantees cleanup
- Multiple resources can be declared
- Resources closed in reverse order

Slide 15: Inheritance and Constructors

Calling parent constructors:

```
class Animal {
    protected String name;

    public Animal(String name) {
        this.name = name;
        System.out.println("Animal constructor: " + name);
    }
}

class Dog extends Animal {
    private String breed;

    public Dog(String name, String breed) {
        super(name); // Must be first statement
        this.breed = breed;
        System.out.println("Dog constructor: " + name + ", " + breed);
    }
}

// Output:
// Animal constructor: Buddy
// Dog constructor: Buddy, Golden Retriever
```

Slide 16: `super()` Constructor Chaining

Constructor call chain in inheritance:

```
class A {
    public A() {
        System.out.println("A");
    }
}

class B extends A {
    public B() {
        System.out.println("B");
    }
}

class C extends B {
    public C() {
        System.out.println("C");
    }
}

// new C() prints:
// A
```



```
// B
// C
```

- Every constructor calls `super()` implicitly (if not specified)
- Chain goes all the way up to `Object`
- Constructors execute from top to bottom

Slide 17: Private Constructors

Restricting object creation:

```
public class Constants {
    // Prevent instantiation
    private Constants() {
        throw new AssertionError("Cannot instantiate");
    }

    public static final double PI = 3.14159;
    public static final double E = 2.71828;
}

// Constants c = new Constants(); // Compile error!
```

Use cases:

- Utility classes (`java.lang.Math`, `java.util.Collections`)
- Singleton pattern
- Factory pattern

Slide 18: Singleton Pattern

Controlled object creation:

```
public class Singleton {
    private static Singleton instance;

    private Singleton() {
        System.out.println("Singleton instance created");
    }

    public static Singleton getInstance() {
        if (instance == null) {
            instance = new Singleton();
        }
        return instance;
    }
}
```

```
// Usage
Singleton s1 = Singleton.getInstance();
Singleton s2 = Singleton.getInstance();
System.out.println(s1 == s2); // true
```

- Private constructor prevents external instantiation
- Single instance managed by class itself
- Lazy initialization on first access

Slide 19: Copy Constructor in Java

Creating a copy of an object:

```
public class Point {
    private int x;
    private int y;

    public Point(int x, int y) {
        this.x = x;
        this.y = y;
    }

    // Copy constructor
    public Point(Point other) {
        this(other.x, other.y);
    }

    @Override
    public String toString() {
        return "Point(" + x + ", " + y + ")";
    }
}

// Usage
Point p1 = new Point(5, 10);
Point p2 = new Point(p1); // Copy
System.out.println(p1);   // Point(5, 10)
System.out.println(p2);   // Point(5, 10)
```

Slide 20: Defensive Copying in Constructors

Protecting against mutation:

```
import java.util.Date;

public class Event {
```

```
private String name;
private Date date; // Mutable object

public Event(String name, Date date) {
    this.name = name;
    // Defensive copy
    this.date = new Date(date.getTime());
}

public Date getDate() {
    // Defensive copy on getter too
    return new Date(date.getTime());
}
}

// Even if caller modifies original date, Event is safe
Date d = new Date();
Event e = new Event("Meeting", d);
d.setTime(0); // Doesn't affect e's date
```

Slide 21: Constructor Validation

Ensuring object validity:

```
public class Person {
    private String name;
    private int age;

    public Person(String name, int age) {
        if (name == null || name.trim().isEmpty()) {
            throw new IllegalArgumentException("Name required");
        }
        if (age < 0 || age > 150) {
            throw new IllegalArgumentException("Invalid age: " + age);
        }
        this.name = name;
        this.age = age;
    }
}

// Person p = new Person("", -5); // IllegalArgumentException!
```

- Fail fast: validate in constructor
- Object is never created in invalid state
- Use checked or unchecked exceptions as appropriate

Slide 22: Cleaner API (Java 9+)

Modern replacement for `finalize()`:

```
import java.lang.ref.Cleaner;

public class DatabaseConnection {
    private static final Cleaner cleaner = Cleaner.create();
    private final Cleaner.Cleanable cleanable;

    public DatabaseConnection(String url) {
        System.out.println("Connected to " + url);

        // Register cleanup action
        cleanable = cleaner.register(this, () -> {
            System.out.println("Cleaning up connection");
        });
    }
}
```

Advantages over `finalize()`:

- Predictable (when `clean()` is called explicitly)
- No performance penalty
- Phantom reference based (not reference counting)
- Can be triggered manually

Slide 23: Constructor vs Factory Method

Aspect	Constructor	Factory Method
Naming	Must match class	Descriptive name
Return	Always new instance	Can cache/reuse
Subclassing	Direct	More flexible
Null return	Not possible	Possible

```
public class Boolean {
    // Cache instances (Flyweight pattern)
    public static final Boolean TRUE = new Boolean(true);
    public static final Boolean FALSE = new Boolean(false);

    private final boolean value;

    private Boolean(boolean value) {
        this.value = value;
    }

    // Factory method - reuses cached instances
}
```

```
public static Boolean valueOf(boolean b) {  
    return b ? TRUE : FALSE;  
}  
}
```

Slide 24: Builder Pattern for Complex Construction

When constructors get too complex:

```
public class Pizza {  
    private String dough;  
    private String sauce;  
    private boolean cheese;  
    private boolean pepperoni;  
    private boolean mushrooms;  
  
    public static class Builder {  
        private String dough = "thin";  
        private String sauce = "tomato";  
        private boolean cheese = true;  
        private boolean pepperoni = false;  
        private boolean mushrooms = false;  
  
        public Builder dough(String val) { dough = val; return this; }  
        public Builder sauce(String val) { sauce = val; return this; }  
        public Builder cheese(boolean val) { cheese = val; return this; }  
        public Builder pepperoni(boolean val) { pepperoni = val; return this; }  
        public Builder mushrooms(boolean val) { mushrooms = val; return this; }  
  
        public Pizza build() {  
            return new Pizza(this);  
        }  
    }  
  
    private Pizza(Builder builder) {  
        this.dough = builder.dough;  
        this.sauce = builder.sauce;  
        this.cheese = builder.cheese;  
        this.pepperoni = builder.pepperoni;  
        this.mushrooms = builder.mushrooms;  
    }  
}  
  
// Usage  
Pizza pizza = new Pizza.Builder()  
    .dough("thick")  
    .pepperoni(true)  
    .mushrooms(true)  
    .build();
```

Slide 25: Record Classes (Java 14+)

Compact constructors with records:

```
public record Point(int x, int y) {  
    // Compact constructor - no field assignment needed  
    public Point {  
        if (x < 0 || y < 0) {  
            throw new IllegalArgumentException("Coordinates cannot be negative");  
        }  
    }  
}  
  
// Usage  
Point p = new Point(5, 10);  
System.out.println(p.x()); // 5  
System.out.println(p.y()); // 10  
System.out.println(p);      // Point[x=5, y=10]
```

- Records automatically generate constructor, getters, equals, hashCode, toString
- Compact constructor syntax for validation
- Immutable by default

Slide 26: Constructor Performance Considerations

Object creation overhead:

```
public class PerformanceDemo {  
    // 1. Direct constructor  
    Object obj = new Object();  
  
    // 2. Class.forName().newInstance() (slow)  
    Class<?> clazz = Class.forName("java.lang.Object");  
    obj = clazz.getDeclaredConstructor().newInstance();  
  
    // 3. clone() (fast, no constructor)  
    // 4. Unsafe.allocateInstance() (bypasses constructor)  
}
```

Object creation cost:

1. Memory allocation (heap)
2. Zeroing memory (default values)
3. Constructor execution
4. `invokespecial` bytecode instruction

Slide 27: Complete Example - Bank Account

```
public class BankAccount implements AutoCloseable {
    private String accountNumber;
    private double balance;
    private boolean closed = false;

    public BankAccount(String accountNumber, double initialBalance) {
        if (accountNumber == null || accountNumber.isEmpty()) {
            throw new IllegalArgumentException("Account number required");
        }
        if (initialBalance < 0) {
            throw new IllegalArgumentException("Negative initial balance");
        }
        this.accountNumber = accountNumber;
        this.balance = initialBalance;
        System.out.println("Account " + accountNumber + " created with $" +
initialBalance);
    }

    public void deposit(double amount) {
        if (closed) throw new IllegalStateException("Account closed");
        if (amount <= 0) throw new IllegalArgumentException("Amount must be
positive");
        balance += amount;
    }

    public boolean withdraw(double amount) {
        if (closed) throw new IllegalStateException("Account closed");
        if (amount > balance) return false;
        balance -= amount;
        return true;
    }

    public double getBalance() { return balance; }

    @Override
    public void close() {
        if (!closed) {
            closed = true;
            System.out.println("Account " + accountNumber + " closed. Final
balance: $" + balance);
        }
    }
}

// Usage with try-with-resources
try (BankAccount acc = new BankAccount("ACC-001", 1000)) {
    acc.deposit(500);
    acc.withdraw(200);
    System.out.println("Balance: $" + acc.getBalance());
}
```


Slide 28: Best Practices for Constructors

Do's:

- ✓ Keep constructors simple (just field initialization)
- ✓ Validate parameters early
- ✓ Use constructor chaining (`this()`, `super()`)
- ✓ Make defensive copies of mutable parameters
- ✓ Prefer `AutoCloseable` over `finalize()`

Don'ts:

- ✗ Call overridable methods in constructors
 - ✗ Perform expensive operations unless necessary
 - ✗ Use `finalize()` in modern Java (Java 9+)
 - ✗ Create unnecessary object references
 - ✗ Let `this` escape during construction
-

Slide 29: Common Mistakes

Mistake 1: Calling overridable methods:

```
public class Parent {
    public Parent() {
        init(); // BAD: overridable method in constructor
    }
    public void init() {
        System.out.println("Parent init");
    }
}

class Child extends Parent {
    private String data = "child data";

    @Override
    public void init() {
        System.out.println(data); // null! (field not yet set)
    }
}
```

Mistake 2: Letting `this` escape:

```
public class BadClass {
    public BadClass() {
        new Thread(() -> System.out.println(this)).start(); // DANGEROUS
    }
}
```

Slide 30: Summary

Java Constructors:

- Special methods that initialize objects
- Can be overloaded with different parameters
- `this()` and `super()` enable constructor chaining
- Private constructors for singleton/utility classes
- Records provide compact constructor syntax

Java "Destructors":

- No explicit destructors (unlike C++)
- `finalize()` is deprecated since Java 9
- `AutoCloseable` + try-with-resources is the modern approach
- `Cleaner` API (Java 9+) for native resource cleanup
- GC handles memory automatically

Key Takeaway:

- Java relies on deterministic cleanup patterns (try-with-resources)
- Constructors are powerful for ensuring object validity
- Resource management is explicit, not implicit

Slide 31: Resources & Further Reading

Official Documentation:

- Java Constructors: docs.oracle.com/javase/tutorial/java/javaOO/constructors.html
- Try-with-resources: docs.oracle.com/javase/tutorial/essential/exceptions/tryResourceClose.html
- Cleaner API: docs.oracle.com/en/java/javase/11/docs/api/java.base/java/lang/ref/Cleaner.html

Topics for Further Study:

- Object pooling patterns
- Serialization and constructors
- Dependency injection constructors
- Immutable object patterns
- Factory pattern vs Builder pattern

Thank You!

Questions?